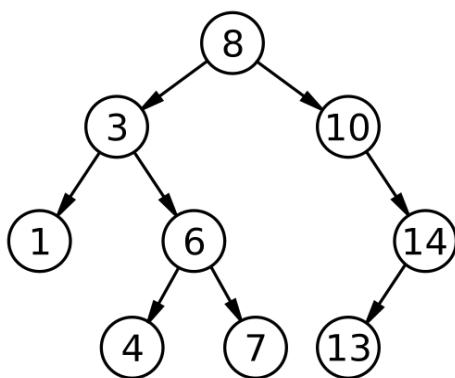


EPFL CIVIL-127, Lab 8

April 14, 2026

8.1

In today's lab, you are going to implement a data structure called a binary search tree. A tree is composed of nodes. Each node holds an integer value, a left child, and a right child. The left and right children are either None or other nodes (i.e. each node has a left and a right subtree). The tree is built in such a way that for every node, all the nodes in the left subtree have values smaller than the node. All the nodes in the right subtree are larger.



Start by implementing a `Node` class. You will want the following methods (and any additional methods that help you):

- `def getValue(self) -> int`
- `def setLeft(self, child: Node | None) -> None`
- `def getLeft(self) -> Node | None`
- `def setRight(self, child: Node | None) -> None`
- `def getRight(self) -> Node | None`

Then implement a `BinarySearchTree` class with the following methods (and any additional methods that you might need):

- `def add(self, value: int) -> None`
 - Adds a new node to the tree with value `value`. If the value already exists in the tree, nothing should happen. New nodes are always added as leaf nodes, the code must therefore traverse the tree until the correct spot is found.
- `def contains(self, value: int) -> bool`
 - Returns true iff the value exists in the tree.
- `def remove(self, value: int) -> None`
 - Removes the node with value `value` from the tree. The left and right subtrees of the node being removed will have to be re-connected to the tree.

One approach is to promote the largest value which is smaller than `value` in its place.

- `def smallest(self) -> int`
 - Returns the smallest value in the tree (i.e. the left most leaf)
- `def largest(self) -> int`
 - Returns the largest value in the tree (i.e. the right most leaf)

Note: `add` and `remove` must maintain the binary search tree invariant (that every node's left subtree has smaller values than the node's value and the right subtree has larger values).

You might have to deal with several different cases (e.g. `remove` behavior is slightly different for leaf vs non-leaf cases), do use a pen and paper to draw the different cases and make sure you have a solid mental model prior to writing any code.

You can raise exceptions when appropriate.

Write unit tests to ensure that your tree works properly. Think about how you are going to test your code before you write any code.

You can also check that the following code gives you the same tree as the illustration above:

```
bst = BinarySearchTree()
bst.add(8)
bst.add(10)
bst.add(14)
bst.add(13)
bst.add(3)
bst.add(6)
bst.add(7)
bst.add(4)
bst.add(1)
```

If you are stuck, read the [Binary search tree](#) wikipedia article. Keep in mind that the article contains spoilers in the form of pseudo-code.

What purpose do binary search trees serve? Binary search trees are mainly used for pedagogical reasons. All the operations you implemented have a worst case complexity of $O(N)$ which isn't desirable in practice. A more useful and improved data structure is the self-balancing binary search tree (e.g. AVL tree), with a worst case complexity of $O(\log N)$ for these operations.